

UNIT 2 · WEEK 2

System calls and low-level I/O

📅 Week 2 · dates to be confirmed </> Lab TD1 (solved + AI challenge) ~ 4 animations

LAB TD1 · SOLVED

Reading and writing with `read` and `write`

The foundations of systems programming: how a program enters the kernel, how to write and read in binary, and why every result must be checked.

EXERCISES

- | | | | |
|---|---|---|----------------------------------|
| 1 | How a system call is implemented | 5 | Copying with <code>read</code> |
| 2 | <code>echo</code> with <code>write</code> | 6 | Reading the integer back |
| 3 | Writing an integer in binary | 7 | Structures in binary and padding |
| 4 | Is <code>perror</code> a system call? | | |

TD1 · 1

● Solved · real output

How a system call is implemented

Statement. A system call needs a special processor instruction (`syscall` on x86-64, `svc` on ARM, `ecall` on RISC-V). Disassemble the C library with `objdump -d` and find how the program tells the kernel *which* call it wants. Then watch with `strace` the calls your `echo` makes.

Solution

Real output

```
$ objdump -d --no-show-raw-insn /usr/lib/x86_64-linux-gnu/libc.so.6 | grep -
A4 -E '^[0-9a-f]+ <getppid@>'
0000000000ff710 <getppid@GLIBC_2.2.5>:
ff710:    endbr64
ff714:    mov     $0x6e,%eax
ff719:    syscall
ff71b:    ret
```

Real output

```
$ objdump -d --no-show-raw-insn /usr/lib/x86_64-linux-gnu/libc.so.6 | grep -
A6 -E '^[0-9a-f]+ <__write_nocancel@@'
0000000000129060 <__write_nocancel@@GLIBC_PRIVATE>:
    129060:    endbr64
    129064:    mov     $0x1,%eax
    129069:    syscall
    12906b:    cmp     $0xffffffffffffffff,%rax
    129071:    ja      129078 <__write_nocancel@@GLIBC_PRIVATE+0x18>
    129073:    ret
```

The call number goes in register `eax` (`110 = getpid`, `1 = write` on x86-64) and the arguments in `rdi`, `rsi`, `rdx`... The `syscall` instruction switches the CPU to kernel mode and jumps to the kernel entry, which uses `eax` as an index into its system-call table. On return the result is in `rax`: a value between `-4095` and `-1` means an error, and the glibc wrapper turns it into `-1 + errno` (that is what `cmp $0xffffffffffffffff,%rax` does).

Real output

```
$ strace -e trace=write ./echo hola mundo > /dev/null
write(1, "hola", 4)                = 4
write(1, " ", 1)                  = 1
write(1, "mundo", 5)              = 5
write(1, "\n", 1)                  = 1
+++ exited with 0 +++
```

Four `write` calls: one per word, one for the space and one for the newline. Each one is a round trip to the kernel; in unit 4 you will see how the standard library groups writes in a buffer to save them.

TD1 · 2

● Solved · tested

echo with write(2)

Statement. Write an `echo` program that copies its arguments to standard output using only `write(2)`. It must check the errors of `write` with `exit_if`. Test it with `./echo blabla < . 1>&0`.

Solution

C tds/td1/echo.c

```
/*
 * TD1 - exercise 3: echo with write(2).
 * Copies its arguments to standard output, separated by spaces, followed by
 * a newline.
 * Try the error path with: ./echo blabla < . 1>&0 (stdout becomes a
 * directory: write fails)
 */
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#include "utils.h"

int main(int argc, char *argv[])
{
    for (int i = 1; i < argc; i++) {
        size_t len = strlen(argv[i]);
        ssize_t rc = write(STDOUT_FILENO, argv[i], len);
        exit_if(rc == -1, "write");
        /* write may write less than asked (pipe or socket full): it is not
         * an error,
         * but this program must then say so. write_all() in libutils loops
         * instead. */
        exit_if((size_t)rc != len, "write: partial write");

        if (i < argc - 1)
            exit_if(write(STDOUT_FILENO, " ", 1) == -1, "write space");
    }
    exit_if(write(STDOUT_FILENO, "\n", 1) == -1, "write newline");
    return EXIT_SUCCESS;
}
```

Real output

```
$ ./echo hola mundo
hola mundo
```

Real output

```
$ ./echo blabla < . 1>&0
write: Bad file descriptor
[exit status 1]
```

`< .` opens the current directory as standard input (descriptor 0) and `1>&0` makes the output (1) a copy of descriptor 0. A directory opened read-only cannot be written: `write` returns `-1` with `errno = EBADF`, and `exit_if` reports it.

TD1 · 3

● Solved · tested

Writing an integer in binary

Statement. Write `write_number`: it writes to standard output the memory content of a `long` equal to 5 and then the number of bytes written by the first call. What do you see in the terminal? Look at the output with `od`.

Solution



C tds/td1/write_number.c

```
/*
 * TD1 - exercise 4: write the memory content of a long, then the number of
 * bytes written.
 * The output is binary, not text: look at it with ./write_number | od -A d
 * -t x1
 */
#include <stdlib.h>
#include <unistd.h>

#include "utils.h"

int main(void)
{
    long value = 5;
    ssize_t written;

    written = write(STDOUT_FILENO, &value, sizeof value);    /* 8 bytes on
x86-64 */
    exit_if(written == -1, "write value");

    exit_if(write(STDOUT_FILENO, &written, sizeof written) == -1, "write
count");
    return EXIT_SUCCESS;
}
```

Real output

```
$ ./write_number | od -A d -t x1
00000000 05 00 00 00 00 00 00 00 08 00 00 00 00 00 00 00
00000016
```

Nothing readable shows up in the terminal: the bytes `05 00 00 00 00 00 00 00` are not printable characters. `od` shows the truth: 8 bytes of the `long` in *little-endian* order (least significant byte first, the way an x86-64 processor stores integers) and 8 bytes holding 8, which is what the first `write` returned. `write` converts nothing to text: it copies bytes.

TD1 · 4

● Solved

Is `perror` a system call?

Answer

No. `perror` is in section 3 of the manual: it is a C library function that reads `errno`, looks up the error text and writes it to standard error... using, internally, the `write` system call. You can check it with `strace ./echo blabla < . 1>&0 : a`
`write(2, "write: Bad file descriptor\n", ...)` shows up.

TD1 · 5

● Solved · tested

Copying input to output with `read(2)`

Statement. Write `read`: it reads standard input with `read` into an array of size `BUFFER_SIZE` and writes it to the output with `write`, until the input is closed (`Ctrl + D`). Test it as `cat`, copying a file and with a directory as input.

Solution

C tds/td1/read.c

```

/*
 * TD1 - exercise 6: copy standard input to standard output with
 * read(2)/write(2).
 * Stops at end of file (Ctrl-D in a terminal). Usages:
 * ./read          ./read < file          ./read < in > out
 * ./read < . (error)
 */
#include <errno.h>
#include <stdlib.h>
#include <unistd.h>

#include "utils.h"

#ifndef BUFFER_SIZE
#define BUFFER_SIZE 4096
#endif

int main(void)
{
    char buffer[BUFFER_SIZE];

    for (;;) {
        ssize_t n = read(STDIN_FILENO, buffer, BUFFER_SIZE);
        if (n == -1 && errno == EINTR)
            continue;
        exit_if(n == -1, "read");
        if (n == 0) /* end of file: the writer closed its
side */
            return EXIT_SUCCESS;

        /* write can be partial: loop until the n bytes are out (see
write_all in libutils). */
        ssize_t done = 0;
        while (done < n) {
            ssize_t rc = write(STDOUT_FILENO, buffer + done, (size_t)(n -
done));
            exit_if(rc == -1, "write");
            done += rc;
        }
    }
}

```

Real output

```
$ ./read < input.txt
first line
second line
```

Real output

```
$ ./read < binary.bin > copy.bin && cmp binary.bin copy.bin && echo identical
identical
```

Real output

```
$ ./read < .
read: Is a directory
[exit status 1]
```



Two details that separate a correct program from one that "almost always" works

- `read` returns `0` at end of file, `-1` on error and otherwise *up to* `BUFFER_SIZE` bytes: it may return fewer (a pipe, a terminal or a signal cut the read short).
- `write` may also write less than requested: that is why the inner loop repeats until the `n` bytes are out. If a signal interrupts it before writing anything, it fails with `EINTR` and a retry is enough.

TD1 · 6

● Solved · tested

Reading the integer of `write_number`

Statement. Write a program that reads what `write_number` produces and prints it with `printf`.

Solution



C tds/td1/read_number.c

```

/*
 * TD1 - exercise 7: read what write_number produces and print it as text.
 * ./write_number | ./read_number      →  value = 5, bytes written = 8
 */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

#include "utils.h"

/* read() may return fewer bytes than asked (e.g. from a pipe): loop until
`len` bytes. */
static void read_exact(int fd, void *buf, size_t len, const char *what)
{
    char *p = buf;
    size_t got = 0;

    while (got < len) {
        ssize_t n = read(fd, p + got, len - got);
        exit_if(n == -1, what);
        exit_if(n == 0, "unexpected end of file");
        got += (size_t)n;
    }
}

int main(void)
{
    long value;
    ssize_t written;

    read_exact(STDIN_FILENO, &value, sizeof value, "read value");
    read_exact(STDIN_FILENO, &written, sizeof written, "read count");
    printf("value = %ld, bytes written = %zd\n", value, written);
    return EXIT_SUCCESS;
}

```

Real output

```

$ ./write_number | ./read_number
value = 5, bytes written = 8

```

`read_exact` repeats `read` until it gets exactly the requested bytes: through a pipe the 16 bytes could arrive in two pieces. It works because both programs run on the same machine: they share the size of `long` and the byte order. Between different machines a format would have to be agreed (for instance network order with `htonl`, unit 8).

TD1 · 7

● Solved · tested

Structures in binary and padding

Statement. Turn the previous programs into `write_struct` and `read_struct` for a `struct nopad { char c1; long l; char c2; }` declared in `utils.h`, holding `'a'`, 5 and `'b'`. How big is the structure? Does it change with `-O3`? How can it be made smaller?

Solution

```
C tds/td1/write_struct.c

/*
 * TD1 - exercise 8: write a structure in binary. struct nopad is declared in
 * utils.h
 * so that write_struct and read_struct agree on its layout.
 */
#include <stdlib.h>
#include <unistd.h>

#include "utils.h"

int main(void)
{
    struct nopad value = { .c1 = 'a', .l = 5, .c2 = 'b' };
    ssize_t written;

    written = write(STDOUT_FILENO, &value, sizeof value);
    exit_if(written == -1, "write value");
    exit_if(write(STDOUT_FILENO, &written, sizeof written) == -1, "write
count");
    return EXIT_SUCCESS;
}
```

C tds/td1/read_struct.c

```

/*
 * TD1 - exercise 8: read the structure written by write_struct.
 * ./write_struct | ./read_struct      → { c1 = 'a', l = 5, c2 = 'b' },
 * bytes written = 24
 */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

#include "utils.h"

static void read_exact(int fd, void *buf, size_t len, const char *what)
{
    char *p = buf;
    size_t got = 0;

    while (got < len) {
        ssize_t n = read(fd, p + got, len - got);
        exit_if(n == -1, what);
        exit_if(n == 0, "unexpected end of file");
        got += (size_t)n;
    }
}

int main(void)
{
    struct nopad value;
    ssize_t written;

    read_exact(STDIN_FILENO, &value, sizeof value, "read value");
    read_exact(STDIN_FILENO, &written, sizeof written, "read count");
    printf("{ c1 = '%c', l = %ld, c2 = '%c' }, bytes written = %zd\n",
           value.c1, value.l, value.c2, written);
    return EXIT_SUCCESS;
}

```

Real output

```

$ ./write_struct | ./read_struct
{ c1 = 'a', l = 5, c2 = 'b' }, bytes written = 24

```

C tds/td1/struct_sizes.c

```

/*
 * TD1 - exercise 8 (answers): size and padding of struct nopad, and two ways
 * to shrink it.
 * The compiler aligns `long` on 8 bytes: 7 padding bytes after c1 and 7
 * after c2 (arrays of
 * the structure must keep every `l` aligned). -O3 does not change the
 * layout: it is part of
 * the ABI, not an optimisation.
 */
#include <stddef.h>
#include <stdio.h>
#include <stdlib.h>

#include "utils.h"

struct reordered {          /* biggest member first: only 6 padding bytes at
the end */
    long l;
    char c1;
    char c2;
};

struct __attribute__((packed)) packed { /* no padding at all (slower,
unaligned access) */
    char c1;
    long l;
    char c2;
};

int main(void)
{
    printf("struct nopad      : size %2zu  offsets c1=%zu l=%zu c2=%zu\n",
sizeof(struct nopad),
        offsetof(struct nopad, c1), offsetof(struct nopad, l),
offsetof(struct nopad, c2));
    printf("struct reordered : size %2zu  offsets l=%zu c1=%zu c2=%zu\n",
sizeof(struct reordered),
        offsetof(struct reordered, l), offsetof(struct reordered, c1),
offsetof(struct reordered, c2));
    printf("struct packed     : size %2zu  offsets c1=%zu l=%zu c2=%zu\n",
sizeof(struct packed),
        offsetof(struct packed, c1), offsetof(struct packed, l),
offsetof(struct packed, c2));
}

```

```
return EXIT_SUCCESS;
}
```

Real output

```
$ ./struct_sizes
struct nopad      : size 24  offsets c1=0 l=8 c2=16
struct reordered  : size 16  offsets l=0 c1=8 c2=9
struct packed     : size 10  offsets c1=0 l=1 c2=9
```

The structure takes **24 bytes** although its fields add up to 10: the compiler aligns `long` on 8 bytes and adds 7 padding bytes after `c1` and 7 at the end (so that, in an array of structures, every `l` stays aligned). `-O3` changes nothing: the memory layout is part of the *ABI*, not an optimisation. To shrink it: order the fields from largest to smallest (16 bytes) or pack it with `__attribute__((packed))` (10 bytes, with slower and non-portable unaligned accesses).



Fixed with respect to the inherited version

`read_struct.c` did not compile: the structure was not in `utils.h` and it used a field `c` that does not exist.

Checklist

Generative AI challenge · A `cat` that measures its own system calls



Goal. Extend `read.c` so that it takes the buffer size as an argument, measure with `strace -c` and with the running time how many `read` and `write` calls it makes for each size, and explain the curve.

Starting prompt. Use it in the tool you prefer (Claude, ChatGPT, Gemini, Copilot, Codex...) with the *solved* version of the lab as context. Ask for a plan first and review every step before accepting it.

Prompt

I have this C program that copies standard input to standard output with read/write and a fixed-size buffer [paste read.c]. Help me: (1) take the buffer size as an argument (1 byte to 1 MiB) using malloc; (2) write a bash script that, for sizes 1, 16, 256, 4096, 65536 and 1048576, copies a 50 MB file, measures the time with `/usr/bin/time -f %e` and counts the calls with `strace -c`; (3) explain why the time drops so much at first and then levels off. Do not give me the conclusion before I see the numbers.

How to check the result

- The copied file is identical (`cmp`) for every size.
- The number of `read` calls is $\approx$ file size / buffer size (plus a final read that returns 0).
- You can explain in your own words the fixed cost of each system call (mode switch).

Declare your use of AI: tool, relevant prompts, what you checked yourself and what you fixed. What counts is that you can explain and verify every decision.